CS 273P: Machine Learning and Data Mining

Homework 5

Due date: **See Canvas**

Instructor: Xiaohui Xie

This homework is graded automatically on **Gradescope** using an autograder. All questions are designed to be unit-testable: you will implement specific functions that return specified outputs that satisfy the autograder. Unless explicitly required, do not print anything.

Allowed libraries: You may use `numpy`, `scipy`, `pandas`, and `PyTorch` (`torch`). You may use `scikit-learn` for `data loading only` (e.g., `sklearn.datasets`). You must implement the core algorithms (k-means, PCA) yourself.

Compute note: The autograder runs on CPU with limited time. Your code must be efficient and deterministic. Use the provided `seed` arguments to ensure deterministic behavior.

Submission rules (READ CAREFULLY):

1. Download the starter files from the GitHub link.
2. Complete your work by editing only the provided Python files: `problem1.py`, `problem2.py`, and `problem3.py`.
3. Create a plain text file called `statement.txt` containing your collaboration statement.
4. Submit to Gradescope by uploading a single **ZIP file** containing exactly:
   - `problem1.py`
   - `problem2.py`
   - `problem3.py`
   - `statement.txt`

   No folders/subdirectories; all files must be at the top level of the zip.
5. You may resubmit before the deadline; your **highest score** counts.

If you run into issues with the autograder or submission format, post on Ed Discussion so others can benefit.

Points: This homework adds up to a total of **110 points**, as follows:

Problem 1: k-means clustering (from scratch)	40 points
Problem 2: PCA for compression and reconstruction (from scratch)	35 points
Problem 3: Denoising autoencoder (PyTorch)	30 points
Statement of Collaboration	5 points

Datasets (Shared)

We will use two small datasets that are easy to work with under CPU constraints:

- **Digits** (8×8 grayscale) from `sklearn.datasets.load_digits()`.
- **Synthetic blobs** from `sklearn.datasets.make_blobs()` for controlled clustering tests.

All tasks must be deterministic given a `seed` argument.

Problem 1: k-means Clustering (40 points)

In this problem you will implement k-means from scratch and evaluate clustering quality.
All required functions are in `problem1.py`.

1.1 Core k-means steps (20 points)

Implement:

1. `init_centroids(X, K, seed, method="kmeans++") -> C` (6 pts)
   Initialize K centroids $C \in \mathbb{R}^{K \times d}$. Support: `"random"` (choose K data points) and `"kmeans++"` initialization. Must be deterministic with `seed`.

2. `assign_clusters(X, C) -> z` (6 pts)
   Assign each point to the nearest centroid (Euclidean distance). Return integer labels $z \in \{0, \ldots, K-1\}^N$.

3. `update_centroids(X, z, K, seed=0) -> C_new` (8 pts)
   Update centroids as cluster means. If a cluster k is empty (no points assigned), reinitialize its centroid by selecting **one** data point uniformly at random from X using the provided `seed` (deterministic). Return $C_{\text{new}} \in \mathbb{R}^{K \times d}$.

1.2 Full algorithm + objective tracking (15 points)

Implement:

1. `kmeans(X, K, seed, max_iters=100, tol=1e-6, method="kmeans++") -> dict` (15 pts)
   Run Lloyd's algorithm. On each iteration: initialize/keep centroids C, assign labels z, then update centroids using `update_centroids(X, z, K, seed)`. Stop when relative improvement in objective is below `tol` or max iterations reached. Return a dict with at least: `{"centroids": C, "labels": z, "inertia": J, "history": list_of_J}` where $J = \sum_i \|x^{(i)} - C_{z_i}\|_2^2$.

1.3 More challenging: clustering evaluation metrics (5 points)

Implement:

1. `cluster_purity(y_true, z_pred) -> float` (5 pts)
   Given ground-truth labels y and cluster assignments z, compute purity: $\frac{1}{N} \sum_k \max_c |\{i : z_i = k, y_i = c\}|$.

Problem 2: PCA for Compression and Reconstruction (35 points)

In this problem you will implement PCA from scratch and use it to compress/reconstruct image vectors.
All required functions are in `problem2.py`.

2.1 PCA via SVD (20 points)

Implement:

1. `pca_fit(X, k) -> dict` (12 pts)
   Given $X \in \mathbb{R}^{N \times d}$, center the data and compute the top-k principal components using SVD. Return a dict with: `{"mu": mu, "components": W, "explained_var": ev}` where $\mu \in \mathbb{R}^d$, $W \in \mathbb{R}^{k \times d}$ (rows are principal directions), and `ev` is a length-k array.

2. `pca_transform(X, model) -> Z` (4 pts)
   Project centered data onto components to get $Z \in \mathbb{R}^{N \times k}$.

3. `pca_inverse(Z, model) -> Xhat` (4 pts)
   Reconstruct back to $\mathbb{R}^{N \times d}$.

2.2 Challenging: choose k by variance retention (10 points)

Implement:

1. `min_k_for_variance(X, var_target=0.9) -> int` (10 pts)
   Return the smallest k such that cumulative explained variance ratio $\geq$ `var_target`. Use PCA computed on X (centered) and be careful about numerical stability.

2.3 Reconstruction quality (5 points)

Implement:

1. `reconstruction_mse(X, Xhat) -> float` (5 pts)
   Return mean squared reconstruction error per entry.

Problem 3: Denoising Autoencoder (30 points)

In this problem you will implement a denoising autoencoder (DAE) in PyTorch and train it on the Digits dataset. All required functions/classes are in `problem3.py`.

3.1 Noise model (8 points)

Implement:

1. `add_noise(x, kind, severity, seed) -> x_noisy` (8 pts)
   Input `x` is a batch tensor of shape (B, d) in $[0, 1]$. Support `kind` in {`"gaussian"`, `"mask"`}:
   - `"gaussian"`: add $\mathcal{N}(0, \sigma^2)$ noise with $\sigma =$`severity`.
   - `"mask"`: randomly set each entry to 0 with probability `severity`.

   Must be deterministic with `seed`. Clip output to $[0, 1]$.

3.2 DAE model (12 points)

Implement:

1. `class DenoisingAutoencoder(nn.Module)` (12 pts)
   A simple fully-connected autoencoder:
   - Encoder: Linear $\rightarrow$ ReLU $\rightarrow$ Linear (latent)
   - Decoder: Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ Sigmoid

   Constructor must accept `input_dim`, `latent_dim`, and `hidden_dim`. Forward takes `x` of shape (B, d) and returns reconstruction $\hat{x}$ of same shape in $[0, 1]$.

3.3 Training + interesting evaluation (10 points)

Implement:

1. `train_dae(model, X_train, noise_kind, severity, seed, epochs=10, batch_size=128, lr=1e-3) -> dict` (7 pts)
   Train the DAE to reconstruct clean inputs from noisy inputs using MSE loss. Return a dict with `"loss_history"` and `"final_loss"`.
2. `denoise_and_score(model, X_test, noise_kind, severity, seed) -> float` (3 pts)
   Add noise to `X_test`, denoise with the model, and return reconstruction MSE.

Challenge goal (informal). A well-implemented DAE should significantly reduce MSE relative to the noisy input under the grading noise settings.

Statement of Collaboration (5 points)

Add a plain text file `statement.txt` to your submission. List the names of collaborators and the nature of collaboration, or write "No collaboration." if you worked alone. Do not share code. Academic honesty policies apply.